The value is a power of two. The default value of `9` represents `2^9 = 512`, a sector size of 512 bytes. To avoid *write amplification* and get the best performance, set this value to the largest sector size used by a device in the pool.

Common drives have 4 KB sectors. Using the default `ashift` of `9` with these drives results in write amplification on these devices. Data contained in a single 4 KB write is instead written in eight 512-byte writes. ZFS tries to read the native sector size from all devices when creating a pool, but drives with 4 KB sectors report that their sectors are 512 bytes for compatibility. Setting `vfs.zfs.min_auto_ashift` to `12` (`2^12 = 4096`) before creating a pool forces ZFS to use 4 KB blocks for best performance on these drives.

Forcing 4 KB blocks is also useful on pools with planned disk upgrades. Future disks use 4 KB sectors, and `ashift` values cannot change after creating a pool.

In some specific cases, the smaller 512-byte block size might be preferable. When used with 512-byte disks for databases or as storage for virtual machines, less data transfers during small random reads. This can provide better performance when using a smaller ZFS record size.

- `vfs.zfs.prefetch.disable` - Disable prefetch. A value of `0` enables and `1` disables it. The default is `0`, unless the system has less than 4 GB of RAM. Prefetch works by reading larger blocks than requested into the ARC in hopes to soon need the data. If the workload has a large number of random reads, disabling prefetch may actually improve performance by reducing unnecessary reads. Adjust this value at any time with sysctl(8).
- `vfs.zfs.vdev.trim_on_init` - Control whether new devices added to the pool have the `TRIM` command run on them. This ensures the best performance and longevity for SSDs, but takes extra time. If the device has already been secure erased, disabling this setting will make the addition of the new device faster. Adjust this value at any time with sysctl(8).
- `vfs.zfs.vdev.max_pending` - Limit the number of pending I/O requests per device. A higher value will keep the device command queue full and may give higher throughput. A lower value will reduce latency. Adjust this value at any time with sysctl(8).
- `vfs.zfs.top_maxinflight` - Upper number of outstanding I/Os per top-level vdev. Limits the depth of the command queue to prevent high latency. The limit is per top-level vdev, meaning the limit applies to each mirror, RAID-Z, or other vdev independently. Adjust this value at any time with sysctl(8).
- `vfs.zfs.l2arc_write_max` - Limit the amount of data written to the L2ARC per second. This tunable extends the longevity of SSDs by limiting the amount of data written to the device. Adjust this value at any time with sysctl(8).
- `vfs.zfs.l2arc_write_boost` - Adds the value of this tunable to `vfs.zfs.l2arc_write_max` and increases the write speed to the SSD until evicting the first block from the L2ARC. This "Turbo Warmup Phase" reduces the performance loss from an empty L2ARC after a reboot. Adjust this value at any time with sysctl(8).
- `vfs.zfs.scrub_delay` - Number of ticks to delay between each I/O during a `scrub`. To ensure that a `scrub` does not interfere with the normal operation of the pool, if any other I/O is happening the `scrub` will delay between each command. This value controls the limit on the total IOPS (I/Os Per Second) generated by the `scrub`. The granularity of the setting is determined by the value of `kern.hz` which defaults to 1000 ticks per second. Changing this setting results in a different

effective IOPS limit. The default value is 4, resulting in a limit of: 1000 ticks/sec / 4 = 250 IOPS. Using a value of *20* would give a limit of: 1000 ticks/sec / 20 = 50 IOPS. Recent activity on the pool limits the speed of `scrub`, as determined by `vfs.zfs.scan_idle`. Adjust this value at any time with sysctl(8).

- `vfs.zfs.resilver_delay` - Number of milliseconds of delay inserted between each I/O during a resilver. To ensure that a resilver does not interfere with the normal operation of the pool, if any other I/O is happening the resilver will delay between each command. This value controls the limit of total IOPS (I/Os Per Second) generated by the resilver. ZFS determins the granularity of the setting by the value of `kern.hz` which defaults to 1000 ticks per second. Changing this setting results in a different effective IOPS limit. The default value is 2, resulting in a limit of: 1000 ticks/sec / 2 = 500 IOPS. Returning the pool to an Online state may be more important if another device failing could Fault the pool, causing data loss. A value of 0 will give the resilver operation the same priority as other operations, speeding the healing process. Other recent activity on the pool limits the speed of resilver, as determined by `vfs.zfs.scan_idle`. Adjust this value at any time with sysctl(8).
- `vfs.zfs.scan_idle` - Number of milliseconds since the last operation before considering the pool is idle. ZFS disables the rate limiting for `scrub` and resilver when the pool is idle. Adjust this value at any time with sysctl(8).
- `vfs.zfs.txg.timeout` - Upper number of seconds between transaction groups. The current transaction group writes to the pool and a fresh transaction group starts if this amount of time elapsed since the previous transaction group. A transaction group may trigger earlier if writing enough data. The default value is 5 seconds. A larger value may improve read performance by delaying asynchronous writes, but this may cause uneven performance when writing the transaction group. Adjust this value at any time with sysctl(8).

22.6.2. ZFS on i386

Some of the features provided by ZFS are memory intensive, and may require tuning for upper efficiency on systems with limited RAM.

22.6.2.1. Memory

As a lower value, the total system memory should be at least one gigabyte. The amount of recommended RAM depends upon the size of the pool and which features ZFS uses. A general rule of thumb is 1 GB of RAM for every 1 TB of storage. If using the deduplication feature, a general rule of thumb is 5 GB of RAM per TB of storage to deduplicate. While some users use ZFS with less RAM, systems under heavy load may panic due to memory exhaustion. ZFS may require further tuning for systems with less than the recommended RAM requirements.

22.6.2.2. Kernel Configuration

Due to the address space limitations of the i386™ platform, ZFS users on the i386™ architecture must add this option to a custom kernel configuration file, rebuild the kernel, and reboot:

```
options        KVA_PAGES=512
```

This expands the kernel address space, allowing the `vm.kvm_size` tunable to push beyond the imposed limit of 1 GB, or the limit of 2 GB for PAE. To find the most suitable value for this option, divide the desired address space in megabytes by four. In this example `512` for 2 GB.

22.6.2.3. Loader Tunables

Increases the kmem address space on all FreeBSD architectures. A test system with 1 GB of physical memory benefitted from adding these options to /boot/loader.conf and then restarting:

```
vm.kmem_size="330M"
vm.kmem_size_max="330M"
vfs.zfs.arc.max="40M"
vfs.zfs.vdev.cache.size="5M"
```

For a more detailed list of recommendations for ZFS-related tuning, see https://wiki.freebsd.org/ZFSTuningGuide.

22.7. Further Resources

- OpenZFS
- FreeBSD Wiki - ZFS Tuning
- Calomel Blog - ZFS Raidz Performance, Capacity and Integrity

22.8. ZFS Features and Terminology

More than a file system, ZFS is fundamentally different. ZFS combines the roles of file system and volume manager, enabling new storage devices to add to a live system and having the new space available on the existing file systems in that pool at once. By combining the traditionally separate roles, ZFS is able to overcome previous limitations that prevented RAID groups being able to grow. A *vdev* is a top level device in a pool and can be a simple disk or a RAID transformation such as a mirror or RAID-Z array. ZFS file systems (called *datasets*) each have access to the combined free space of the entire pool. Used blocks from the pool decrease the space available to each file system. This approach avoids the common pitfall with extensive partitioning where free space becomes fragmented across the partitions.

pool	A storage *pool* is the most basic building block of ZFS. A pool consists of one or more vdevs, the underlying devices that store the data. A pool is then used to create one or more file systems (datasets) or block devices (volumes). These datasets and volumes share the pool of remaining free space. Each pool is uniquely identified by a name and a GUID. The ZFS version number on the pool determines the features available.

vdev Types	A pool consists of one or more vdevs, which themselves are a single disk or a group of disks, transformed to a RAID. When using a lot of vdevs, ZFS spreads data across the vdevs to increase performance and maximize usable space. All vdevs must be at least 128 MB in size. • *Disk* - The most basic vdev type is a standard block device. This can be an entire disk (such as /dev/ada0 or /dev/da0) or a partition (/dev/ada0p3). On FreeBSD, there is no performance penalty for using a partition rather than the entire disk. This differs from recommendations made by the Solaris documentation.  Using an entire disk as part of a bootable pool is strongly discouraged, as this may render the pool unbootable. Likewise, you should not use an entire disk as part of a mirror or RAID-Z vdev. Reliably determining the size of an unpartitioned disk at boot time is impossible and there's no place to put in boot code. • *File* - Regular files may make up ZFS pools, which is useful for testing and experimentation. Use the full path to the file as the device path in `zpool create`. • *Mirror* - When creating a mirror, specify the `mirror` keyword followed by the list of member devices for the mirror. A mirror consists of two or more devices, writing all data to all member devices. A mirror vdev will hold as much data as its smallest member. A mirror vdev can withstand the failure of all but one of its members without losing any data.  To upgrade a regular single disk vdev to a mirror vdev at any time, use `zpool attach`. • *RAID-Z* - ZFS uses RAID-Z, a variation on standard RAID-5 that offers better distribution of parity and eliminates the "RAID-5 write hole" in which the data and parity information become inconsistent after an unexpected restart. ZFS supports three levels of RAID-Z which provide varying levels of redundancy in exchange for decreasing levels of usable storage. ZFS uses RAID-Z1 through RAID-Z3 based on the number of parity devices in the array and the number of disks which can fail before the pool stops being operational. In a RAID-Z1 configuration with four disks, each 1 TB, usable storage is 3 TB and the pool will still be able to operate in degraded mode with one faulted disk. If another disk goes offline before replacing and resilvering the faulted disk would result in losing all pool data. In a RAID-Z3 configuration with eight disks of 1 TB, the volume will provide 5 TB of usable space and still be able to operate with three faulted disks. Sun™ recommends no more than nine disks in a single vdev. If more disks make up the configuration, the recommendation is to divide them into separate vdevs and stripe the pool data across them. A configuration of two RAID-Z2 vdevs consisting of 8 disks each would create something like a RAID-60 array. A RAID-Z group's storage capacity is about the size of

Transaction Group (TXG)	Transaction Groups are the way ZFS groups blocks changes together and writes them to the pool. Transaction groups are the atomic unit that ZFS uses to ensure consistency. ZFS assigns each transaction group a unique 64-bit consecutive identifier. There can be up to three active transaction groups at a time, one in each of these three states: * *Open* - A new transaction group begins in the open state and accepts new writes. There is always a transaction group in the open state, but the transaction group may refuse new writes if it has reached a limit. Once the open transaction group has reached a limit, or reaching the `vfs.zfs.txg.timeout`, the transaction group advances to the next state. * *Quiescing* - A short state that allows any pending operations to finish without blocking the creation of a new open transaction group. Once all the transactions in the group have completed, the transaction group advances to the final state. * *Syncing* - Write all the data in the transaction group to stable storage. This process will in turn change other data, such as metadata and space maps, that ZFS will also write to stable storage. The process of syncing involves several passes. On the first and biggest, all the changed data blocks; next come the metadata, which may take several passes to complete. Since allocating space for the data blocks generates new metadata, the syncing state cannot finish until a pass completes that does not use any new space. The syncing state is also where *synctasks* complete. Synctasks are administrative operations such as creating or destroying snapshots and datasets that complete the uberblock change. Once the sync state completes the transaction group in the quiescing state advances to the syncing state. All administrative functions, such as `snapshot` write as part of the transaction group. ZFS adds a created synctask to the open transaction group, and that group advances as fast as possible to the syncing state to reduce the latency of administrative commands.
Adaptive Replacement Cache (ARC)	ZFS uses an Adaptive Replacement Cache (ARC), rather than a more traditional Least Recently Used (LRU) cache. An LRU cache is a simple list of items in the cache, sorted by how recently object was used, adding new items to the head of the list. When the cache is full, evicting items from the tail of the list makes room for more active objects. An ARC consists of four lists; the Most Recently Used (MRU) and Most Frequently Used (MFU) objects, plus a ghost list for each. These ghost lists track evicted objects to prevent adding them back to the cache. This increases the cache hit ratio by avoiding objects that have a history of occasional use. Another advantage of using both an MRU and MFU is that scanning an entire file system would evict all data from an MRU or LRU cache in favor of this freshly accessed content. With ZFS, there is also an MFU that tracks the most frequently used objects, and the cache of the most commonly accessed blocks remains.

L2ARC	L2ARC is the second level of the ZFS caching system. RAM stores the primary ARC. Since the amount of available RAM is often limited, ZFS can also use cache vdevs. Solid State Disks (SSDs) are often used as these cache devices due to their higher speed and lower latency compared to traditional spinning disks. L2ARC is entirely optional, but having one will increase read speeds for cached files on the SSD instead of having to read from the regular disks. L2ARC can also speed up deduplication because a deduplication table (DDT) that does not fit in RAM but does fit in the L2ARC will be much faster than a DDT that must read from disk. Limits on the data rate added to the cache devices prevents prematurely wearing out SSDs with extra writes. Until the cache is full (the first block evicted to make room), writes to the L2ARC limit to the sum of the write limit and the boost limit, and afterwards limit to the write limit. A pair of sysctl(8) values control these rate limits. `vfs.zfs.l2arc_write_max` controls the number of bytes written to the cache per second, while `vfs.zfs.l2arc_write_boost` adds to this limit during the "Turbo Warmup Phase" (Write Boost).
ZIL	ZIL accelerates synchronous transactions by using storage devices like SSDs that are faster than those used in the main storage pool. When an application requests a synchronous write (a guarantee that the data is stored to disk rather than merely cached for later writes), writing the data to the faster ZIL storage then later flushing it out to the regular disks greatly reduces latency and improves performance. Synchronous workloads like databases will profit from a ZIL alone. Regular asynchronous writes such as copying files will not use the ZIL at all.
Copy-On-Write	Unlike a traditional file system, ZFS writes a different block rather than overwriting the old data in place. When completing this write the metadata updates to point to the new location. When a shorn write (a system crash or power loss in the middle of writing a file) occurs, the entire original contents of the file are still available and ZFS discards the incomplete write. This also means that ZFS does not require a fsck(8) after an unexpected shutdown.
Dataset	*Dataset* is the generic term for a ZFS file system, volume, snapshot or clone. Each dataset has a unique name in the format *poolname/path@snapshot*. The root of the pool is a dataset as well. Child datasets have hierarchical names like directories. For example, *mypool/home*, the home dataset, is a child of *mypool* and inherits properties from it. Expand this further by creating *mypool/home/user*. This grandchild dataset will inherit properties from the parent and grandparent. Set properties on a child to override the defaults inherited from the parent and grandparent. Administration of datasets and their children can be delegated.
File system	A ZFS dataset is most often used as a file system. Like most other file systems, a ZFS file system mounts somewhere in the systems directory hierarchy and contains files and directories of its own with permissions, flags, and other metadata.
Volume	ZFS can also create volumes, which appear as disk devices. Volumes have a lot of the same features as datasets, including copy-on-write, snapshots, clones, and checksumming. Volumes can be useful for running other file system formats on top of ZFS, such as UFS virtualization, or exporting iSCSI extents.

Snapshot	The copy-on-write (COW) design of ZFS allows for nearly instantaneous, consistent snapshots with arbitrary names. After taking a snapshot of a dataset, or a recursive snapshot of a parent dataset that will include all child datasets, new data goes to new blocks, but without reclaiming the old blocks as free space. The snapshot contains the original file system version and the live file system contains any changes made since taking the snapshot using no other space. New data written to the live file system uses new blocks to store this data. The snapshot will grow as the blocks are no longer used in the live file system, but in the snapshot alone. Mount these snapshots read-only allows recovering of previous file versions. A rollback of a live file system to a specific snapshot is possible, undoing any changes that took place after taking the snapshot. Each block in the pool has a reference counter which keeps track of the snapshots, clones, datasets, or volumes use that block. As files and snapshots get deleted, the reference count decreases, reclaiming the free space when no longer referencing a block. Marking snapshots with a hold results in any attempt to destroy it will returns an `EBUSY` error. Each snapshot can have holds with a unique name each. The release command removes the hold so the snapshot can deleted. Snapshots, cloning, and rolling back works on volumes, but independently mounting does not.
Clone	Cloning a snapshot is also possible. A clone is a writable version of a snapshot, allowing the file system to fork as a new dataset. As with a snapshot, a clone initially consumes no new space. As new data written to a clone uses new blocks, the size of the clone grows. When blocks are overwritten in the cloned file system or volume, the reference count on the previous block decreases. Removing the snapshot upon which a clone bases is impossible because the clone depends on it. The snapshot is the parent, and the clone is the child. Clones can be *promoted*, reversing this dependency and making the clone the parent and the previous parent the child. This operation requires no new space. Since the amount of space used by the parent and child reverses, it may affect existing quotas and reservations.
Checksum	Every block is also checksummed. The checksum algorithm used is a per-dataset property, see `set`. The checksum of each block is transparently validated when read, allowing ZFS to detect silent corruption. If the data read does not match the expected checksum, ZFS will attempt to recover the data from any available redundancy, like mirrors or RAID-Z. Triggering a validation of all checksums with `scrub`. Checksum algorithms include: * `fletcher2` * `fletcher4` * `sha256` The `fletcher` algorithms are faster, but `sha256` is a strong cryptographic hash and has a much lower chance of collisions at the cost of some performance. Deactivating checksums is possible, but strongly discouraged.

Compression	Each dataset has a compression property, which defaults to off. Set this property to an available compression algorithm. This causes compression of all new data written to the dataset. Beyond a reduction in space used, read and write throughput often increases because fewer blocks need reading or writing. * *LZ4* - Added in ZFS pool version 5000 (feature flags), LZ4 is now the recommended compression algorithm. LZ4 works about 50% faster than LZJB when operating on compressible data, and is over three times faster when operating on uncompressible data. LZ4 also decompresses about 80% faster than LZJB. On modern CPUs, LZ4 can often compress at over 500 MB/s, and decompress at over 1.5 GB/s (per single CPU core). * *LZJB* - The default compression algorithm. Created by Jeff Bonwick (one of the original creators of ZFS). LZJB offers good compression with less CPU overhead compared to GZIP. In the future, the default compression algorithm will change to LZ4. * *GZIP* - A popular stream compression algorithm available in ZFS. One of the main advantages of using GZIP is its configurable level of compression. When setting the `compress` property, the administrator can choose the level of compression, ranging from `gzip1`, the lowest level of compression, to `gzip9`, the highest level of compression. This gives the administrator control over how much CPU time to trade for saved disk space. * *ZLE* - Zero Length Encoding is a special compression algorithm that compresses continuous runs of zeros alone. This compression algorithm is useful when the dataset contains large blocks of zeros.
Copies	When set to a value greater than 1, the `copies` property instructs ZFS to maintain copies of each block in the crossref:zfs[zfs-term-filesystem,file system] or volume. Setting this property on important datasets provides added redundancy from which to recover a block that does not match its checksum. In pools without redundancy, the copies feature is the single form of redundancy. The copies feature can recover from a single bad sector or other forms of minor corruption, but it does not protect the pool from the loss of an entire disk.

Deduplication	Checksums make it possible to detect duplicate blocks when writing data. With deduplication, the reference count of an existing, identical block increases, saving storage space. ZFS keeps a deduplication table (DDT) in memory to detect duplicate blocks. The table contains a list of unique checksums, the location of those blocks, and a reference count. When writing new data, ZFS calculates checksums and compares them to the list. When finding a match it uses the existing block. Using the SHA256 checksum algorithm with deduplication provides a secure cryptographic hash. Deduplication is tunable. If `dedup` is `on`, then a matching checksum means that the data is identical. Setting `dedup` to `verify`, ZFS performs a byte-for-byte check on the data ensuring they are actually identical. If the data is not identical, ZFS will note the hash collision and store the two blocks separately. As the DDT must store the hash of each unique block, it consumes a large amount of memory. A general rule of thumb is 5-6 GB of ram per 1 TB of deduplicated data). In situations not practical to have enough RAM to keep the entire DDT in memory, performance will suffer greatly as the DDT must read from disk before writing each new block. Deduplication can use L2ARC to store the DDT, providing a middle ground between fast system memory and slower disks. Consider using compression instead, which often provides nearly as much space savings without the increased memory.
Scrub	Instead of a consistency check like fsck(8), ZFS has `scrub`. `scrub` reads all data blocks stored on the pool and verifies their checksums against the known good checksums stored in the metadata. A periodic check of all the data stored on the pool ensures the recovery of any corrupted blocks before needing them. A scrub is not required after an unclean shutdown, but good practice is at least once every three months. ZFS verifies the checksum of each block during normal use, but a scrub makes certain to check even infrequently used blocks for silent corruption. ZFS improves data security in archival storage situations. Adjust the relative priority of `scrub` with `vfs.zfs.scrub_delay` to prevent the scrub from degrading the performance of other workloads on the pool.
Dataset Quota	ZFS provides fast and accurate dataset, user, and group space accounting as well as quotas and space reservations. This gives the administrator fine grained control over space allocation and allows reserving space for critical file systems. ZFS supports different types of quotas: the dataset quota, the reference quota (refquota), the user quota, and the group quota. Quotas limit the total size of a dataset and its descendants, including snapshots of the dataset, child datasets, and the snapshots of those datasets.  Volumes do not support quotas, as the `volsize` property acts as an implicit quota.
Reference Quota	A reference quota limits the amount of space a dataset can consume by enforcing a hard limit. This hard limit includes space referenced by the dataset alone and does not include space used by descendants, such as file systems or snapshots.
User Quota	User quotas are useful to limit the amount of space used by the specified user.

Group Quota	The group quota limits the amount of space that a specified group can consume.
Dataset Reservation	The `reservation` property makes it possible to guarantee an amount of space for a specific dataset and its descendants. This means that setting a 10 GB reservation on storage/home/bob prevents other datasets from using up all free space, reserving at least 10 GB of space for this dataset. Unlike a regular `refreservation`, space used by snapshots and descendants is not counted against the reservation. For example, if taking a snapshot of storage/home/bob, enough disk space other than the `refreservation` amount must exist for the operation to succeed. Descendants of the main data set are not counted in the `refreservation` amount and so do not encroach on the space set. Reservations of any sort are useful in situations such as planning and testing the suitability of disk space allocation in a new system, or ensuring that enough space is available on file systems for audio logs or system recovery procedures and files.
Reference Reservation	The `refreservation` property makes it possible to guarantee an amount of space for the use of a specific dataset *excluding* its descendants. This means that setting a 10 GB reservation on storage/home/bob, and another dataset tries to use the free space, reserving at least 10 GB of space for this dataset. In contrast to a regular reservation, space used by snapshots and descendant datasets is not counted against the reservation. For example, if taking a snapshot of storage/home/bob, enough disk space other than the `refreservation` amount must exist for the operation to succeed. Descendants of the main data set are not counted in the `refreservation` amount and so do not encroach on the space set.
Resilver	When replacing a failed disk, ZFS must fill the new disk with the lost data. *Resilvering* is the process of using the parity information distributed across the remaining drives to calculate and write the missing data to the new drive.
Online	A pool or vdev in the `Online` state has its member devices connected and fully operational. Individual devices in the `Online` state are functioning.
Offline	The administrator puts individual devices in an `Offline` state if enough redundancy exists to avoid putting the pool or vdev into a Faulted state. An administrator may choose to offline a disk in preparation for replacing it, or to make it easier to identify.
Degraded	A pool or vdev in the `Degraded` state has one or more disks that disappeared or failed. The pool is still usable, but if other devices fail, the pool may become unrecoverable. Reconnecting the missing devices or replacing the failed disks will return the pool to an Online state after the reconnected or new device has completed the Resilver process.
Faulted	A pool or vdev in the `Faulted` state is no longer operational. Accessing the data is no longer possible. A pool or vdev enters the `Faulted` state when the number of missing or failed devices exceeds the level of redundancy in the vdev. If reconnecting missing devices the pool will return to an Online state. Insufficient redundancy to compensate for the number of failed disks loses the pool contents and requires restoring from backups.

Chapter 23. Other File Systems

23.1. Synopsis

File systems are a fundamental component of any operating system. They enable users to store, manage, and access data, making storage devices like hard drives, flash drives, and USB storage devices practical for everyday use. Different operating systems use different file systems natively.

FreeBSD has traditionally used the Unix File System (UFS), with the modernized UFS2 as its primary native file system. FreeBSD also uses the Z File System (ZFS), known for its advanced features, robustness, and reliability. See The Z File System (ZFS) for more details.

In addition to its native file systems, FreeBSD supports a broad range of file systems from other operating systems. Support for these file systems varies, some require loading kernel modules, while others need additional userland tools.

Before reading this chapter:

- Be familiar with UNIX® concepts and FreeBSD basics.
- Feel comfortable installing software in FreeBSD.
- Have some understanding of disks, storage devices, and FreeBSD's naming conventions for devices.

Read this chapter to:

- Understand the differences between native and supported file systems.
- Know which file systems are supported by FreeBSD and the level of support available.
- Learn how to enable, configure, access, and work with non-native file systems.

23.2. Linux® File Systems

FreeBSD provides built-in support for several Linux® file systems. This section demonstrates how to load support for and how to mount the supported Linux® file systems.

23.2.1. Extended File System (EXT)

Kernel support for Extended File System (EXT) file systems has been available since FreeBSD 2.2. The ext2fs(4) driver allows the FreeBSD kernel to both read and write to ext2, ext3, and ext4 file systems.



Journalling and encryption are not supported yet.

To access an ext file system, mount the ext volume by specifying its FreeBSD partition name and an existing mount point. This example mounts /dev/ada1s1 on /mnt:

```
# mount -t ext2fs /dev/ada1s1 /mnt
```

23.3. Windows® File Systems

FreeBSD supports FAT, exFAT, and NTFS file systems, enabling access to Windows-formatted storage.

23.3.1. FAT File System

The FAT file system is a simple and robust file system. Although it lacks the performance, reliability, and scalability of modern alternatives, its availability on many different operating systems makes it a common choice for data exchange between devices.

To access a FAT file system, mount the FAT volume by specifying its FreeBSD partition name and an existing mount point. This example mounts /dev/ada0s1 on /mnt:

```
# mount -t msdosfs /dev/ada0s1 /mnt
```

23.3.2. exFAT File System

exFAT (Extended File Allocation Table) is a lightweight file system optimized for flash storage devices, such as USB drives and SD cards. It supports large file sizes and is widely used across various platforms, making it ideal for external storage.

To use exFAT on FreeBSD, install the filesystems/exfat package, load the FUSE kernel module, and mount the file system as shown below:

Install the exFAT package:

```
# pkg install fusefs-exfat
```

Before using a FUSE file system, load the fusefs(4) kernel module:

```
# kldload fusefs
```

Use sysrc(8) to load the module at startup:

```
# sysrc kld_list+=fusefs
```

Mount the exFAT volume by specifying its FreeBSD partition name and an existing mount point. This example mounts /dev/ada0s1 on /mnt:

```
# mount.exfat /dev/ada0s1 /mnt
```

23.3.3. NTFS File System

NTFS is a robust file system developed by Microsoft® and commonly used in Windows operating systems. FreeBSD provides full read and write support for NTFS through the filesystems/ntfs package, making it easy to access and modify NTFS-formatted storage devices.

To use NTFS on FreeBSD, install the filesystems/ntfs package, load the FUSE kernel module, and mount the file system as shown below:

Install the NTFS package:

```
# pkg install fusefs-ntfs
```

Before using a FUSE file system, load the fusefs(4) kernel module:

```
# kldload fusefs
```

Use sysrc(8) to load the module at startup:

```
# sysrc kld_list+=fusefs
```

Mount the NTFS volume by specifying its FreeBSD partition name and an existing mount point. This example mounts /dev/ada0s1 on /mnt:

```
# ntfs-3g /dev/ada0s1 /mnt
```

23.4. MacOS® File Systems

FreeBSD provides support for MacOS® file systems, including HFS/HFS+, allowing access to storage devices formatted for Apple® systems.

23.4.1. HFS/HFS+ File System

HFS/HFS+ was the primary file system for MacOS prior to APFS, commonly used on older Mac devices and external drives. FreeBSD provides read-only support for HFS/HFS+ through the filesystems/hfsfuse package.

To use HFS/HFS+ on FreeBSD, install the filesystems/hfsfuse package, load the FUSE kernel module, and mount the file system as shown below:

Install the HFS/HFS+ package:

```
# pkg install fusefs-hfsfuse
```

Before using a FUSE file system, load the fusefs(4) kernel module:

```
# kldload fusefs
```

Use sysrc(8) to load the module at startup:

```
# sysrc kld_list+=fusefs
```

Mount the HFS/HFS+ volume by specifying its FreeBSD partition name and an existing mount point. This example mounts /dev/ada0s1 on /mnt:

```
# hfsfuse /dev/ada0s1 /mnt
```

Chapter 24. Virtualization

24.1. Synopsis

Virtualization software allows multiple operating systems to run simultaneously on the same computer. Such software systems for PCs often involve a host operating system which runs the virtualization software and supports any number of guest operating systems.

Read this chapter to learn:

- The difference between a host operating system and a guest operating system.
- How to install FreeBSD on the following virtualization platforms:
  - Parallels Desktop(Apple® macOS®)
  - VMware Fusion(Apple® macOS®)
  - VirtualBox™(Microsoft® Windows®, Intel®-based Apple® macOS®, Linux)
  - QEMU(FreeBSD)
  - bhyve(FreeBSD)
- How to tune a FreeBSD system for best performance under virtualization.

Before reading this chapter:

- Understand the basics of UNIX® and FreeBSD.
- Know how to install FreeBSD.
- Know how to set up a network connection.
- Know how to install additional third-party software.

24.2. FreeBSD as a Guest on Parallels Desktop for macOS®

Parallels Desktop for Mac® is a commercial software product available for Apple® Mac® computers running macOS® 10.14.6 or higher. FreeBSD is a fully supported guest operating system. Once Parallels has been installed on macOS®, the user must configure a virtual machine and then install the desired guest operating system.

24.2.1. Installing FreeBSD on Parallels Desktop on Mac®

The first step in installing FreeBSD on Parallels is to create a new virtual machine for installing FreeBSD.

Choose **Install Windows or another OS from a DVD or image file** and proceed.